

Analyzing Parallelism and Performance in PyTorch

Master Project

University of Basel
Faculty of Science
Department of Mathematics and Computer Science
High Performance Computing Group

Advisor and Examiner:
Prof. Dr. Florina M. Ciorba
Supervisor: Reto Krummenacher

Scott Barcomb
scott.barcomb@stud.unibas.ch
24-061-640

January 19, 2026



Abstract

This report describes the profiling and analysis of a PyTorch deep learning workload through the use of various profiling tools. The main goal is to understand how PyTorch exploits hardware parallelism across CPUs and GPUs. This was done by profiling the fine-tuning of the Apertus eight-billion parameter Instruct model. Profiling results show that PyTorch employs a wide range of parallelism mechanisms, from CPU-side Python- and library-based multiprocessing for data loading and tensor operations, to asynchronous GPU execution. Device parallelism was also evaluated with data parallelism techniques to understand how PyTorch reacts to training scalability and what communication primitives must come into play. Applying the insights gained to a reinforcement learning game-based application could not be completed due to time and hardware constraints. Had that goal been reached, extra insight on PyTorch performance from a second deep learning perspective could have been discussed.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	3
1.3	Contribution	3
1.4	Outline	4
2	Background: Execution Models and Parallelism in PyTorch	5
2.1	PyTorch Execution Model and Abstraction Layers	5
2.2	Forms of Parallelism in PyTorch	5
2.3	Asynchronous GPU Execution and Synchronization	6
2.4	Data Loading and Multiprocessing	6
2.5	Profiling Deep Learning Workloads	6
2.6	Memory and Computation Trade-offs in Large Models	6
3	Methodology	7
3.1	Experimental Platform	7
3.2	Model and Training Configuration	7
3.3	Profiling and Monitoring Tools	7
3.4	Experiments	7
3.4.1	Baseline Measurement	9
3.4.2	Input Pipeline Sensitivity	9
3.4.3	Batch Size and Sequence Length Scaling	9
3.4.4	Gradient Accumulation Trade-offs	9
3.4.5	Activation Checkpointing	9
3.4.6	Two-GPU Data Parallel Fine Tuning with LoRA	9
3.4.7	Two-GPU Data Parallel Full-Parameter Fine Tuning	9
3.5	Reproducibility	10
4	Results and Discussion	11
4.1	Baseline	11
4.1.1	Baseline Performance	11
4.1.2	Profiling Trace Breakdown	12
4.2	DataLoader Parallelism	15
4.2.1	Quantitative Analysis	15
4.2.2	Profiling Analysis	16
4.3	Sequence Length and Batch Size on Throughput and Memory	17
4.4	Gradient Accumulation	18
4.5	Activation Checkpointing	19
4.6	Two-GPU Data Parallel Fine-Tuning with LoRA	20
4.6.1	First Experiment	20
4.6.2	Second Experiment	21
4.7	Full-Parameter Fine-Tuning with Optimizer Offloading	22
5	Conclusion	23
	Bibliography	24

List of Acronyms

HPC	High Performance Computing
I/O	Input/Output
miniHPC	HPC cluster of the HPC-Group at the Department of Mathematics and Computer Science
OpenMP	A shared-memory multi-platform multiprocessing programming interface supported on C, C++, and Fortran
MKL	Math Kernel Library for compute intensive tasks on Intel CPUs
SLURM	Simple Linux Utility for Resource Management
LoRA	Low-Rank Adaptation of large language models
ZeRO	Zero Redundancy Optimizer for training large language models
ZeRO-2	Shards gradient and optimizer states across GPUs, reducing memory pressure
ZeRO-3	Shards parameter states on top of what ZeRO-2 shreds for larger-scale training, at the cost of more communication overhead

Chapter 1

Introduction

1.1 Motivation

Deep learning frameworks like PyTorch are important tools for machine learning because they offer users an easy, flexible solution for AI model engineering. That being said, the performance of deep learning workloads is dependent on efficient hardware parallelization through the use of multi-core CPUs, GPUs, and distributed systems. What is left is a divide between ease-of-use and performance, which creates a challenge for users who wish to understand the execution of PyTorch applications or optimize them.

PyTorch relies on an execution stack involving native libraries (OpenMP, Intel MKL), asynchronous CUDA kernel execution, and distributed communication primitives, and most of this parallelism is hidden from the user. As a result, performance bottlenecks related to kernel scheduling, synchronization, memory usage, or data loading can be difficult to diagnose or even reason about.

The central research question for this project is thus: how does PyTorch exploit hardware parallelism, and how does it affect the performance of training a deep learning workload? Existing work on PyTorch performance mostly covers specific subsystems, such as data loading, or benchmarking against other deep learning frameworks. This motivates the attempt to connect the high-level execution model of PyTorch with a profiling of real runtime behavior.

1.2 Objectives

Through the use of profiling, this project aims to:

- Identify where and how parallelism happens within a PyTorch training step.
- Analyze how synchronization and memory management affect performance.
- Evaluate how optimization techniques such as batch size, gradient accumulation, activation checkpointing, and distributed training affect execution behavior.
- Relate observations back to known parallelism mechanisms.

1.3 Contribution

This project provides two main contributions: a structured analysis of the different forms of parallelism within PyTorch and an in-depth profiling study of large language model fine-tuning. The aim is to highlight performance bottlenecks on the hardware system being evaluated, and their relationship to discussed parallelism mechanisms.

As a case study, this work focuses on fine-tuning the Apertus large language model, in particular the eight billion-parameter Instruct model, on the miniHPC cluster. A few profiling tools are used to capture execution traces and resource utilization data, including the PyTorch profiler and system utilities for CPU and GPU monitoring.

An additional project objective involved implementing and optimizing a reinforcement learning model for a game-based application. Due to time constraints and unavailability of hardware, this objective could

not be completed and is therefore not included in the experiment report. Nevertheless, the insights shown in this paper should provide a comprehensive overview of PyTorch performance characteristics.

1.4 Outline

Chapter 2 gives background on PyTorch’s execution model and the different forms of parallelism it employs. In Chapter 3, the experimental design and workload configuration is discussed. Chapter 4 presents experimental results with discussion, analyzing performance behavior under various training configurations. Finally, Chapter 5 concludes the report with a summary of findings.

Chapter 2

Background: Execution Models and Parallelism in PyTorch

2.1 PyTorch Execution Model and Abstraction Layers

Users of PyTorch are given a Python-based programming model with which they can easily design and modify neural networks. While this design is great for flexibility, it obscures the actual execution design of a layered runtime system made up of high-level Python operations mapped onto backend libraries. As a result, the structure of the Python program does not correspond to the underlying hardware computation.

At runtime, the Python control flow invokes tensor operations which are defined in the ATen library, a native PyTorch library. These operators subsequently work on backend implementations that rely on multi-threaded CPU libraries such as OpenMP or Intel MKL, or schedule GPU kernel launches via CUDA. GPU execution in particular is asynchronous, often executing large swathes of kernels enqueued onto a stream by the host CPU, only stopping when synchronization is required. As was revealed later in this project, this decoupling greatly improves hardware utilization but also complicated analysis since the timing and order of computation are not immediately evident within traces.

This execution model is described in the paper by Paszke et al. [1] as a key design choice for giving PyTorch its flexibility and performance. These abstraction layers that give PyTorch its performance, however, are also what make understanding its performance-critical work so hard do.

2.2 Forms of Parallelism in PyTorch

There are multiple forms of parallelism used by PyTorch that manifest at different stages of execution. Rather than treat these forms independently, it would be better to consider how they arise within a training step during deep learning and how they influence performance at that stage.

Inter-step parallelism arises from overlapping data loading between training steps with device execution during a step. The DataLoader method from PyTorch supports Python multiprocessing to enable data batch preparation while the model is still executing on the GPU [2]. This overlap can greatly increase performance in workloads with high-latency data loading pipelines.

Intra-step parallelism occurs within a single training step, or the execution of a forward and backward pass during normal training. This type of parallelism includes multi-threading for tensor operations on the CPU via OpenMP, and launches of CUDA kernels on the GPU.

Cross-device parallelism refers to distributed training across multiple GPUs. The simplest way to accomplish this is through PyTorch’s DistributedDataParallel (DDP) method, which synchronizes model gradients across GPUs using collective communication operations [3]. By synchronizing gradients, models are effectively replicated across devices, which does not relieve memory constraints imposed by models too large to train on a single GPU. For that purpose, approaches such as DeepSpeed’s ZeRO (Zero Redundancy Optimizer) instead shard gradients, optimizer states, and other model parameters across devices to reduce the per-device memory requirement [4]. What this results in, however, is a huge increase in communication and synchronization calls to ensure training is scaling properly, resulting in serious runtime overheads for workloads where model sharding is not necessary.

2.3 Asynchronous GPU Execution and Synchronization

CUDA kernel launches are nonblocking and execution asynchronous in PyTorch. This allows the CPU host to queue work on the GPU and continue to execute more Python afterwards, allowing overlap between GPU and host work. While great for performance, this model also complicates profiling and interpretation. GPU utilization metrics alone cannot distinguish between productive computation and time spent on synchronization or communication, and so secondary metrics such as throughput and training runtime need to be taken. Similarly, while profiling traces of the CPU and GPU can depict the kernel launch patterns, these kernels can be queued and launched extremely rapidly, and there can be dozens or even hundreds in a single training step, making it hard to visually quantify their execution pattern. Therefore, in order to accurately analyze the performance of PyTorch’s asynchronous execution, a combination of these tools needs to be used, in combination with manipulating techniques that can change synchronization behavior such as gradient accumulation and activation checkpointing.

2.4 Data Loading and Multiprocessing

PyTorch’s `DataLoader` supports multiprocessing for data, but its effectiveness in speeding up training is highly dependent on the workload; for workloads that require a lot CPU preprocessing and have high storage latency, data loading can become a bottleneck.

In a paper by Svogor et al. [5], a detailed analysis of `DataLoader` was conducted. Their work demonstrated scenarios in which input pipelines would dominate training time if not properly managed, which highlights the importance of profiling data loading separately to understand the impact worker counts and configurations can have on training workloads.

That being said, for compute-intensive workloads like the Apertus case-study in this project, the impact of data loading can be negligibly small. Therefore, while not explicitly seeking end-to-end training performance gains, an inspection of profiling traces for PyTorch multiprocessing, and an evaluation of data loading performance can be conducted.

2.5 Profiling Deep Learning Workloads

Applications like PyTorch can be difficult to profile due to various discussed reasons such as asynchronous execution and multiple layers of software abstraction. PyTorch’s `torch.profiler` can provide insight into these layers by highlighting what PyTorch operators get called, what library calls these operators make, and the GPU kernel launches that subsequently spring from them. In addition, various system tools such as `nvidia-smi` and `htop` can provide resource utilization metrics for GPU and CPU execution.

There exist benchmarking suites such as TorchBench [6] that aim to compare deep learning frameworks like PyTorch across models and platforms, but they do not provide insight into the execution specifics of these platforms. This project is instead using profiling as an investigative tool rather than a benchmarking instrument.

2.6 Memory and Computation Trade-offs in Large Models

Training large models can take a significant amount of memory to accomplish, which has produced techniques such as activation checkpointing, gradient accumulation, and parameter sharding, all three of which are implemented in the Apertus transformers model. Activation checkpointing reduces memory usage by discarding the outputs of intermediate activation functions in the forward pass, in favor of recomputing them during the backward pass. This effectively reduces the memory footprint of the model during step-by-step training at the cost of increased computational load. Gradient accumulation holds off on updating model weights until after a number of steps have passed in training, accumulating the gradients along the way. This mimics training with a larger batch size while reducing memory pressure.

ZeRO-based optimizers reduce memory consumption by sharding (or partitioning) optimizer states, gradients, and parameters across devices. These techniques allow for previously unworkable training configurations to become possible, at the cost of increasingly larger communication overhead as more of the model is sharded. Therefore, a balance has to be struck between memory savings and communication cost, which can depend on the number of devices and the training workload. In this project, an examination of these trade-offs is conducted in both LoRA-based and full-parameter fine-tuning.

Chapter 3

Methodology

3.1 Experimental Platform

All experiments were conducted on the miniHPC GPU node at the University of Basel which contains two NVIDIA A100 PCIe GPUs (each with 40 GB of VRAM) and two Intel Xeon Gold 6258 CPUs

The system runs on CentOS 7.9. To ensure reproducibility and compatibility with PyTorch and its dependencies, a Miniconda-based Python environment was used to manage all packages for this project.

Due to network restrictions on compute nodes, the Apertus 8B Instruct model weights and the Multilingual Thinking dataset were downloaded on the login node ahead of time, since these were originally pulled from the HuggingFace hub during execution.

3.2 Model and Training Configuration

The primary workload analyzed in this project was the fine-tuning of the Apertus 8B Instruct large language model using Low-Rank Adaptation (LoRA). LoRA was selected because it reduces memory requirements for fine-tuning, allowing profiling experiments to comfortably fit on a single GPU and scaled to two GPUs using data parallelism.

Fine-tuning was performed using the official Apertus fine-tuning script, which is implemented in PyTorch and makes use of the HuggingFace Transformers library. Default training parameters provided by the configuration files in the Apertus Fine-Tuning Recipes repository were used to establish a stable baseline.

3.3 Profiling and Monitoring Tools

A combination of profiling tools were used to capture several aspects of execution behavior:

- `torch.profiler` was used to collect detailed profiling traces, which depict CPU and GPU activities such as python methods, native library methods, and GPU kernels.
- `nvidia-smi` was used to monitor GPU utilization, memory consumption, and power.
- `htop` was used to observe CPU utilization during data loading experiments.

3.4 Experiments

In order to study different sources of performance impacts, a series of experiments were conducted that focused on specific aspects of PyTorch’s execution behavior. The full experimental plan is summarized in Tables 3.1 and 3.2.

For all experiments, `torch.profiler` traces were collected using a schedule with a warmup of two steps, a wait period of ten steps, and four recorded training steps.

Table 3.1: Experimental setup for profiling Apertus fine-tuning on miniHPC.

Factor	Value	Properties
Application	Apertus fine-tuning	Training modes: LoRA (primary), full fine-tuning (feasibility study).
Metrics	Performance and resource usage	Average runtime (Apertus-reported), throughput (samples/s), average GPU utilization, average GPU memory usage.
Profiling	Instrumentation	<code>torch.profiler</code> (operator- and kernel-level traces); <code>nvidia-smi</code> for GPU utilization and memory statistics; <code>htop</code> for CPU utilization in DataLoader experiments.
Computing system	miniHPC node	$2 \times$ Intel Xeon Gold 6258 CPUs; $2 \times$ NVIDIA A100-PCIE-40GB GPUs.
Validity	Repetitions	5 runs per configuration

Table 3.2: Overview of conducted profiling experiments.

Experiment	GPUs	Training	Varied parameters
Baseline	1	LoRA	Configuration defaults
Input pipeline sensitivity	1	LoRA	<code>num_workers</code> $\in \{0, 2, 6, 12\}$
Batch/sequence scaling	1	LoRA	Batch size $\times$ sequence length grid
Gradient accumulation	1	LoRA	<code>grad_accum</code> $\in \{1, 2, 4, 8\}$
Checkpointing impacts	1	LoRA	Activation checkpointing $\in \{\text{on}, \text{off}\}$
2-GPU parallelization	2	LoRA + data parallel	World size = 2; best single-GPU configuration
Full fine-tuning feasibility	2	Full + data parallel	Best configuration from prior experiments

3.4.1 Baseline Measurement

The baseline experiment established reference values for training throughput, peak GPU memory usage, and execution behavior. A single GPU with default training and data loading configurations were used, with one change. The maximum token sequence length was reduced from 4096 to 1024 due to GPU memory constraints. Additionally, training was performed with Low-Rank Adaptation (LoRA) enabled.

The anatomy of a training step was also examined here to determine the relationship between PyTorch functions and native library calls, and how GPU kernel-launches work.

3.4.2 Input Pipeline Sensitivity

To evaluate the impact of data loading on training performance, an analysis of the PyTorch DataLoader method was performed. The following parameters were varied:

- Number of worker processes ($num_workers \in \{0, 2, 6, 12\}$)

Profiling traces were examined for the length of data-fetching operation between training steps, and CPU utilization was collected to assess worker efficiency. The goal of this experiment was to identify the impact data loading had on the workload and to find configurations that minimized input pipeline delay.

3.4.3 Batch Size and Sequence Length Scaling

For this experiment, the impact of batch size and token sequence length on GPU memory use were examined. GPU utilization and memory consumption were measured to check how these parameters scale GPU work, and profiling traces were taken to see how the GPU stream was affected.

3.4.4 Gradient Accumulation Trade-offs

Gradient accumulation was evaluated as a strategy for increasing effective batch size without exceeding memory limits. Accumulation steps of $\{1, 2, 4, 8\}$ were tested. GPU resource utilization was measured to see how what affects gradient accumulation steps had on memory pressure and training throughput.

3.4.5 Activation Checkpointing

Activation checkpointing was evaluated for GPU memory usage and execution traces were analyzed to understand this technique’s effect on VRAM savings and additional compute overhead due to recomputation.

3.4.6 Two-GPU Data Parallel Fine Tuning with LoRA

To evaluate distributed training behavior, the best-performing single-GPU configuration identified in prior experiments was extended to two GPUs using PyTorch’s DistributedDataParallel (DDP) as well as ZeRO-2 and ZeRO-3, which are model sharding techniques created by DeepSpeed (2.2). Both GPUs were profiled to assess communication overhead and load balance between devices, and the changes in training runtime compared to the single-GPU experiments.

3.4.7 Two-GPU Data Parallel Full-Parameter Fine Tuning

The feasibility of full fine-tuning on a two-GPU system was investigated as a final experiment. ZeRO-2 and ZeRO-3 model sharding techniques were applied to offload the model’s optimizer states to the CPU, allowing the training run to fit within memory. Both GPUs were profiled to again assess communication overhead and load balancing, and training throughput was monitored to assess training speed relative to two-GPU LoRA training.

This experiment focuses on the feasibility of full-parameter fine-tuning in a memory-constrained environment and discusses the tradeoffs and costs associated with it.

3.5 Reproducibility

All experiments were conducted under controlled conditions on the same hardware. Configuration files, the profiling script, and training parameter values were kept consistent across runs when applicable. To recreate the experiment, these same files can be used in conjunction with the project Miniconda environment and SLURM batch job.

Chapter 4

Results and Discussion

4.1 Baseline

4.1.1 Baseline Performance

To begin with, confirmation that Apertus could run on the cluster was needed, and for that purpose, a baseline fine-tuning run was performed using the default Apertus LoRA configuration with one modification. This modification was necessary to avoid CUDA out-of-memory errors on the NVIDIA A100 GPU that was used, and involved lowering the maximum token sequence length from 4096 to 1024. To get an idea of what we were working with, average GPU utilization during training over one epoch was recorded with `nvidia-smi`, with data samples being taken every second.

Figure 4.1 shows snapshot from `nvidia-smi` taken during training. Boxed in red shows the current-moment GPU memory usage during training, and this level remained mostly stable throughout. What this means is that the LoRA fine-tuning setup was able to fit comfortably within the VRAM limits of the GPU, and it left room for future experiments using larger batch sizes, longer sequence lengths, and other features like turning off activation checkpointing.

NVIDIA-SMI 545.23.06			Driver Version: 545.23.06			CUDA Version: 12.3		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	NVIDIA A100-PCIE-40GB	P0	Off	00000000:1B:00.0	Off	85%	0	0
N/A	53C		246W / 250W	24815MiB / 40960MiB			Default	Disabled
1	NVIDIA A100-PCIE-40GB	P0	Off	00000000:B2:00.0	Off	0%	0	0
N/A	38C		56W / 250W	4MiB / 40960MiB			Default	Disabled
Processes:								
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage		
ID	ID	ID						
0	N/A	N/A	4876	C	...e/mamba/envs/apertus/bin/python3.10	24802MiB		

Figure 4.1: A snapshot of Apertus LoRA fine-tuning taken during training. At the moment, approximately 24.8 GB of GPU memory is being used. Additionally, power draw on the device is close to the limit, at 246 watts.

GPU utilization for this run averaged about 89.5%, which immediately signaled the GPU-intensive nature of the workload. Lower utilization would have indicated stalled work due to CPU-side bottlenecks. That being said, there is no way to tell from `nvidia-smi` alone whether the GPU is doing productive training work the whole time. For that purpose, in the following experiments, finer-grained execution behavior such as kernel activity and synchronization points will be discussed using traces captured by the PyTorch profiling tool. Table 1 shows the peak VRAM usage and average GPU utilization of this experiment during steady-state training.

Metric	Baseline
Peak VRAM usage	24.8 GB
Avg GPU utilization	89.5%
GPUs used	1

Table 4.1: The baseline performance profile of Apertus LoRA-based fine-tuning captured with nvidia-smi.

4.1.2 Profiling Trace Breakdown

Exported chrome trace files taken by the PyTorch profiler break execution into two main sections: one for the main Python thread executed on the CPU, and one for the stream of CUDA kernels executed on the GPU.

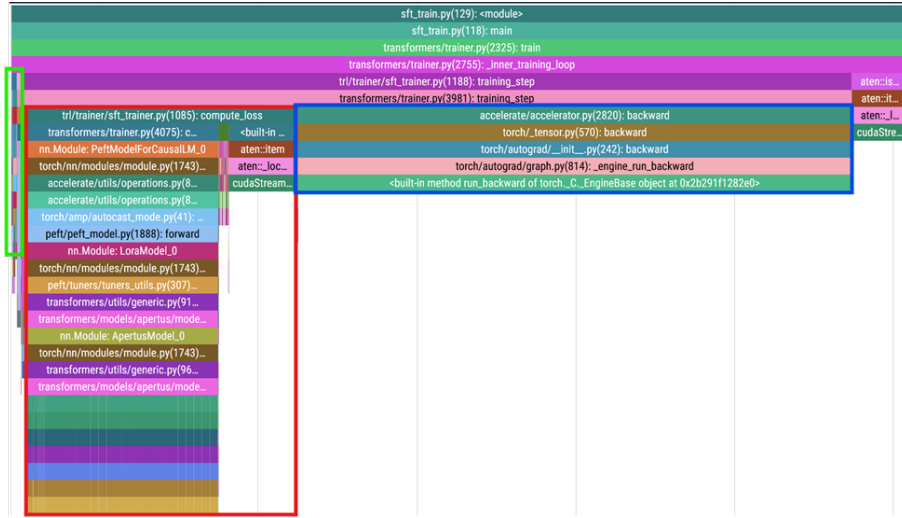


Figure 4.2: One training step of Apertus LoRA fine-tuning on the CPU. Boxed in green on the far left: input pipeline boundary. Data loading for the training step is being done here and takes up a very small portion of the step. Boxed in red on the left: forward pass. Boxed in blue in the middle: backward pass.

Figure 4.2 depicts a view of a single training step for Apertus LoRA. Visible are the stack of high-level Python operations being executed. Traversing through the trace from left to right represents the flow of execution time; in this example, one step took approximately 3 seconds to execute. Traversing through the stack vertically from top to bottom represents higher-level methods calling lower-level ones. Eventually, these methods decompose into native library operations, which in turn launch GPU-executed kernels.

In Figure 4.3, we continue to traverse down the stack that represents the forward pass, until ATen (PyTorch native tensor library) operators are visible. These handle tensor operations inherent to deep learning training workflows and are the actual operators that launch kernels on the GPU.

Zooming back out, we can see a view of the CUDA kernel stream being executed on the GPU over one training step (Figure 4.4). The many multi-colored thin bands represent individual kernels, demonstrating how many hundreds or thousands are launched to successfully complete execution of the forward and backward region viewed in the trace on the CPU side.

Figure 4.5 depicts the asynchronous nature of PyTorch GPU scheduling. The kernel launch-calls that are boxed in red do not directly correlate to the kernels currently being executed on the GPU in the same time frame. Instead, these kernels are scheduled to be executed, letting the CPU continue to work until synchronization is necessary (Figures 4.6 and 4.7).



Figure 4.3: The bottom of the forward pass operator stack on the CPU exposed by the torch profiler. Four tensor operations are being performed in order, with heavy compute work being sourced to the GPU via `cudaLaunchKernel`. The stack is so tall because many small kernels implement one conceptual layer of training like the forward pass.



Figure 4.4: A view of the entire CUDA kernel stream on the GPU during one training step. The multicolored bands represent individual kernels that were executed.

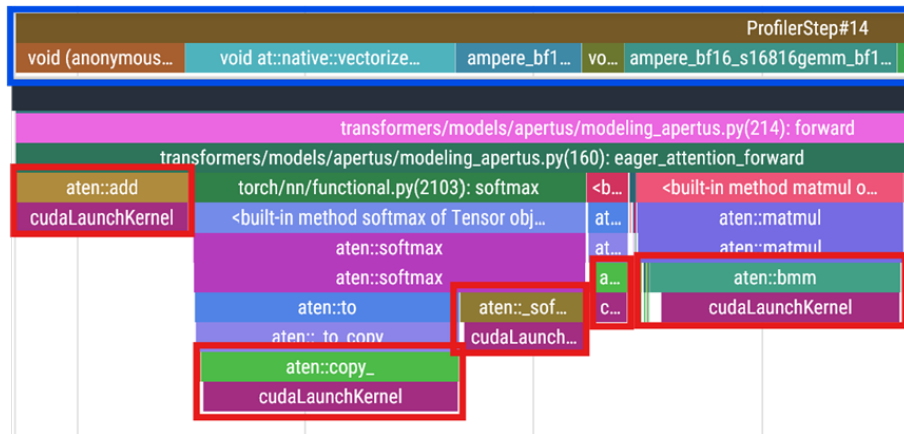


Figure 4.5: A side-by-side view of the GPU trace (top, boxed in blue) captured within the same time window as the CPU trace (bottom, CUDA kernel launches boxed in red). At this moment, the CPU is currently working on the configured attention implementation (eager attention) during the forward pass. The GPU is executing previously scheduled kernels asynchronously.

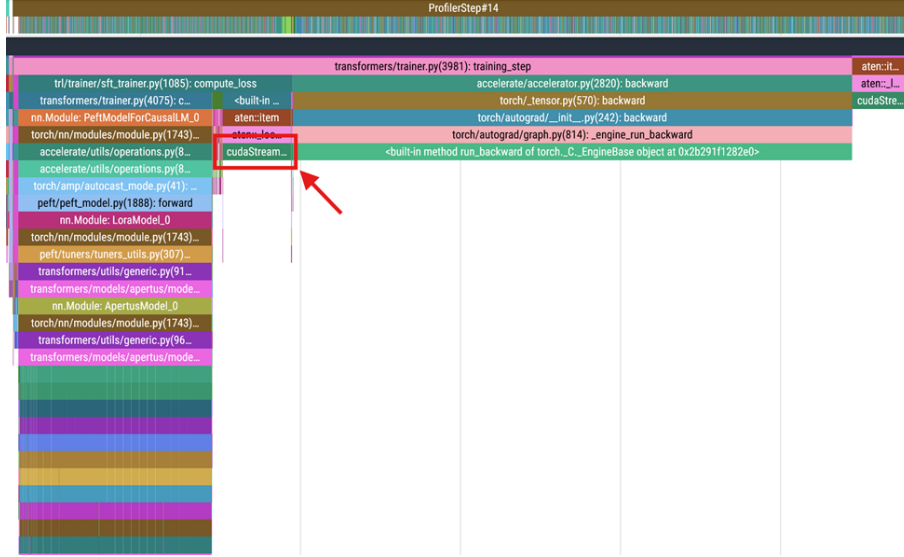


Figure 4.6: A depiction of the full training step on the CPU, like in Figure 2, but this time with the GPU kernel stream pinned to the top for clarity. Boxed in red is a CUDA stream synchronization point that was called after the forward pass.

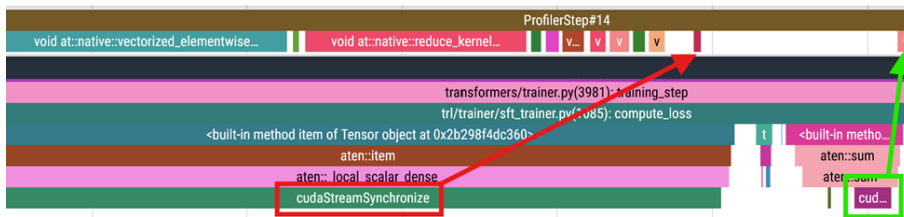


Figure 4.7: A zoomed-in view depicting the end of the synchronization point shown in Figure 4.6. The red arrow points to where a gap in GPU work corresponds to synchronization. The green arrow points to where GPU work picks up again with the first kernel launch after synchronization.

4.2 DataLoader Parallelism

This experiment evaluated the impact of varying the PyTorch DataLoader’s `num_workers` parameter on training performance and CPU utilization. Four configurations were tested: $num_workers \in \{0, 2, 6, 12\}$. For each setup, the total runtime, its training throughput in samples per second, and the average CPU utilization over 1-, 5-, and 15-minute intervals were recorded near the end of the training epoch.

4.2.1 Quantitative Analysis

Training throughput and runtime remained mostly the same across all configurations. There was about a 0.3% difference in runtime between each configuration, and throughput was stable at approximately 1.55 samples/s, indicating that DataLoader parallelism did not improve training performance in any meaningful way.

Worker Count	CPU Util Percentage Avg 1min, 5min, 15min	Runtime (s)	Samples/s
0	82%, 76%, 68%	644.81	1.551
2	99%, 86%, 73%	646.01	1.548
6	98%, 89%, 78%	645.94	1.548
12	92%, 84%, 79%	646.40	1.547

Table 4.2: Effect of DataLoader worker count on CPU utilization and training performance.

In contrast, the CPU utilization experienced noticeable changes between runs. Moving from 0 to 2 workers immediately increased the short-term CPU utilization, coinciding with the introduction of multiprocessing for data loading. This higher utilization level was also seen with the higher worker counts of 6 and 12, with the addition of elevated 15-minute averages compared to the two-worker configuration. These increases, however, did not translate into noticeable performance gains.

To note, these experiments were not run back-to-back. Ample time padding was given for the CPU utilization averages – which were reported by `htop` – to fall back to idle levels when no jobs were being run on the node. The raised 15-minute averages for the higher worker counts could reflect higher minute-to-minute CPU utilization due to the extra Python processes that were spawned.

Overall, these results suggest that the training loop is not input pipeline bound. Data loading work is negligibly small relative to GPU computation, and so increasing parallelism CPU-side did not result in any real reduction of the critical part of the training step.

That being said, increasing the number of workers did increase the effective speed with which the input pipeline fetched data between training steps, as highlighted in Table 4.3. The average runtime of `torch/utils/data/dataloader.py`’s `__next__` function was recorded over four training steps for each configuration. This function retrieves the next batch of samples and formats it for the Apertus model.

Worker Count	Avg Runtime of DataLoader <code>__next__</code>
0	2ms 620 μ s
2	1ms 561 μ s
6	0ms 845 μ s
12	0ms 345 μ s

Table 4.3: Effect of DataLoader worker count on batch data retrieval speed.

An immediate and consistent speedup in batch fetching was noticed when increasing the number of workers, suggesting that if given the proper amount of PyTorch intra-op threads to work with, PyTorch is able to effectively take advantage of Python multiprocessing for speeding up data loading. While the gained speedup is not very significant when considering each training step has a duration on the order of seconds, not milliseconds, the PyTorch DataLoader could still be useful for eking out faster training times over many epochs for GPU-intensive workloads like Apertus.

4.2.2 Profiling Analysis

Torch Profiler traces were captured for each configuration as a verification of these results. While training speed remained stable, the profiler showed clear differences in CPU-side execution.

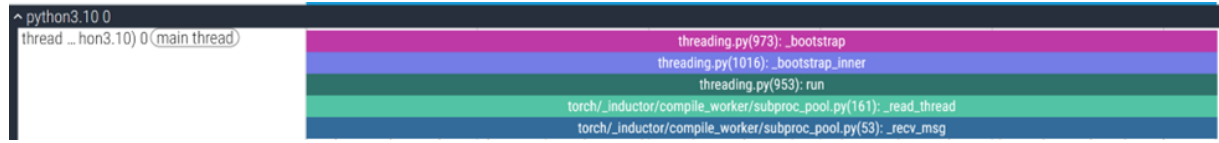


Figure 4.8: The main Python multiprocessing thread as seen in Torch profiling traces. Each horizontal bar represents a Python function that was called by the bar stacked above it.

With 0 workers, CPU execution was led by the main Python thread, and data loading occurred synchronously within the training loop (Figure 4.8). As the worker counts were increased, the traces displayed an increasing number of background threads associated with the DataLoader worker pool, including repeated activity in Python’s multiprocessing queues and thread synchronization calls (waits). As can be seen in Figure 4.9, the CPU trace became significantly deeper for higher worker counts, reflecting the increased coordination and waiting overhead among the worker processes.

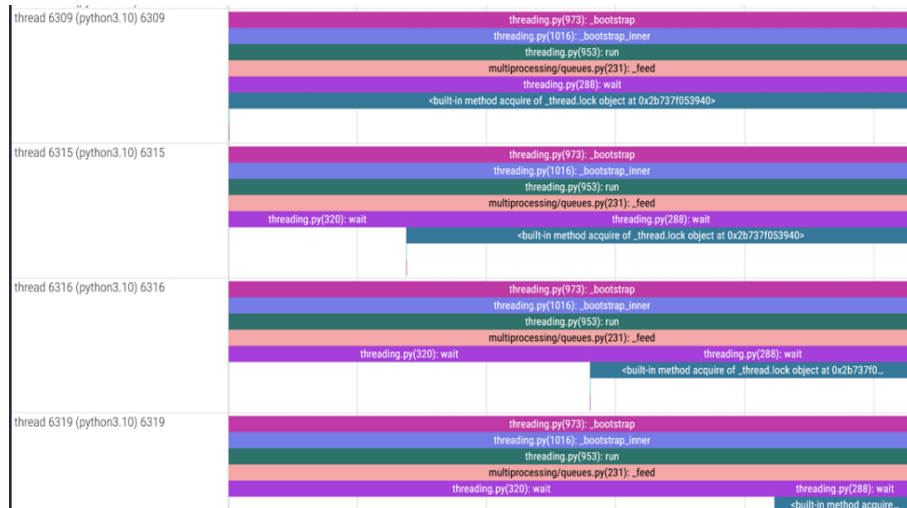


Figure 4.9: With 6 workers, multiple Python threads become visible in the profiling trace.

Even with this increased CPU activity, the CUDA execution timeline did not change between runs. As can be seen in Figure 4.10, GPU kernels were launched regularly without visible gaps, which indicated the GPU was continuously fed with data and that data loading was not a bottleneck.

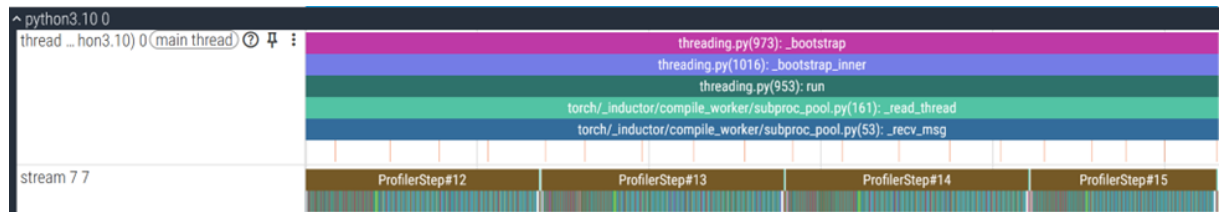


Figure 4.10: A more widely cropped view of Figure 4.8 showing the CUDA kernel stream over four training steps. The thin, multi-colored bands are individual kernels being executed on the GPU. The very thin, white gaps in the kernel stream at the end of each step represent time where the GPU is not working, due to CPU synchronization or I/O stalls.

Python multiprocessing can be seen in action within the trace capturing the main CPU training loop too. Although only taking up a fractionally small portion of each training step, as seen in Figure 4.2, you can zoom in to see that when there are zero workers, all data fetching, formatting, and collating for model training is handled within the main thread (Figure 4.11). Meanwhile, at higher worker counts like

12, this work is outsourced to the worker processes by a call to multiprocessing/queues.py’s put function (Figure 4.12). This suggests that in workloads where data loading and preprocessing takes a sizable chunk of training step time, increasing the number of DataLoader workers could introduce significant training speedups.

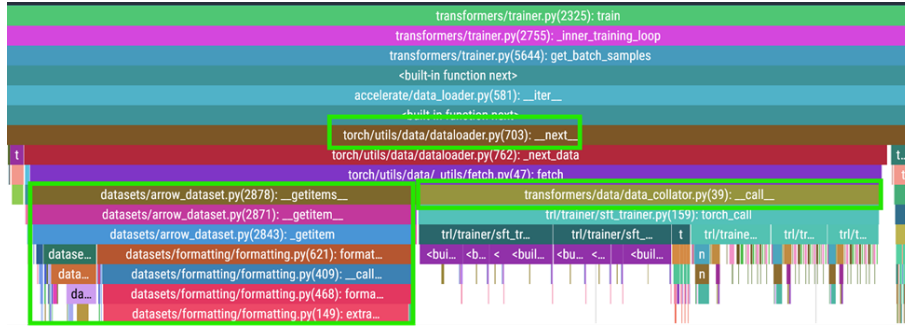


Figure 4.11: The case where 0 workers are helping the main Python thread with data loading during training. Before each training step, the main thread must fetch the data batch and preprocess it on its own.

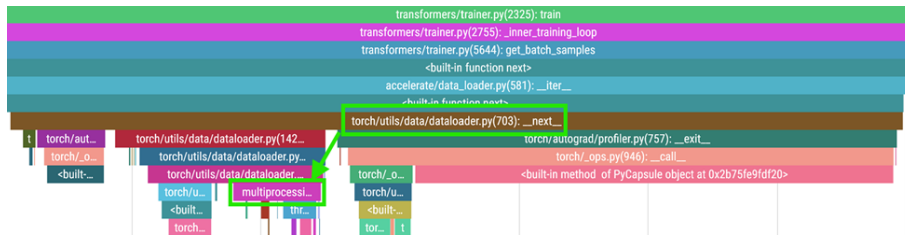


Figure 4.12: The case where there are workers on hand to help the main Python thread with data loading. The main thread can simply queue the work needed to be done on the multiprocessing queue.

4.3 Sequence Length and Batch Size on Throughput and Memory

Seq Len $\times$ Batch Size	GPU Util (% , Mem)	Runtime	Samples/s	Loss
1024 $\times$ 4	98.5%, 24.8 GB	646.68	1.546	1.082
1024 $\times$ 8	98.9%, 33.3 GB	696.85	1.435	1.112
1024 $\times$ 16	CUDA OOM			
2048 $\times$ 1	79.5%, 19.4 GB	591.43	1.691	1.134
2048 $\times$ 2	96.2%, 22.4 GB	623.35	1.604	1.089
2048 $\times$ 4	98.4%, 29.1 GB	759.99	1.316	1.075
2048 $\times$ 8	CUDA OOM			
4096 $\times$ 1	79.5%, 19.3 GB	599.08	1.669	1.134
4096 $\times$ 2	99.1%, 22.4 GB	635.64	1.573	1.089
4096 $\times$ 4	CUDA OOM			

Table 4.4: Effect of sequence length and batch size on GPU utilization, runtime, throughput, and loss.

Table 4.4 summarizes the impact of varying sequence length and batch size on the training performance of Apertus fine-tuning. GPU utilization remained high across the tested configurations, and memory consumption scaled with both sequence length and batch size. Several configurations quickly blew up in size and resulted in CUDA out-of-memory (OOM) errors, which demonstrated the impact of memory constraints on sequence length and batch size selection.

For a maximum sequence length of 1024 tokens, increasing the batch size from 4 to 8 increased GPU memory usage from 24.8 GB to 33.3 GB and decreased throughput from 1.546 to 1.435 samples/s. The

runtime also increased, which indicated diminishing returns from purely increasing batch size. Increasing batch size to 16 was impossible due to OOM occurring. Similarly, at a sequence length of 2048, batch sizes 1 and 2 reported the best throughput while batch size 4 showed a noticeable drop despite its near-identical peak GPU utilization; batch size 8 again resulted in OOM.

For the longest tested sequence length of 4096 tokens, only batch sizes 1 and 2 were feasible. These configurations were again comparable in their throughput to the 2048-token runs, but with no observable gains from increasing batch size beyond 2. In all cases, loss values remained stable, which indicated the performance differences came from aspects of the training environment rather than any variation in training quality.

Torch profiler traces collected for a few steps in each run provided further insight into these results. Increasing batch size or sequence length primarily increased kernel duration, which improved GPU saturation up to a point, before simply extending the duration of GPU runtime. This can be seen in Figures 4.13, 4.14, and 4.15, which depict a 100-millisecond snapshot of the CUDA stream during a training step for Apertus LoRA fine-tuning. For these runs, a sequence length of 2048 with batch sizes of 1, 2, and 4 were used, in order. In Figure 4.13, there are noticeable gaps in the CUDA stream due to the short duration of the CUDA kernels, which become more filled-in in Figure 4.14 where a batch size of 2 is used, representing an efficient increase in the GPU utilization. Figure 4.15, however, depicts overcrowding of the CUDA stream. There are no visible gaps, indicating that the longer kernel times are causing additional kernels to wait and are reducing overall GPU efficiency. This explains why GPU utilization remained close to maximum for all workloads, but the training speed in the third case was noticeably lower.



Figure 4.13: The GPU kernel stream taken over a 100 millisecond duration during the forward pass with a sequence length of 2048 tokens and a batch size of 1.



Figure 4.14: The GPU kernel stream taken over a 100 millisecond duration during the forward pass with a sequence length of 2048 tokens and a batch size of 2.



Figure 4.15: The GPU kernel stream taken over a 100 millisecond duration during the forward pass with a sequence length of 2048 tokens and a batch size of 4.

Overall, these results indicated that for Apertus fine-tuning on the tested hardware, moderate batch sizes combined with longer sequence lengths does not necessarily improve throughput and both parameters quickly become memory bound and introduce GPU slowdowns. Slightly shorter sequences with smaller batch sizes provided the best balance between feasibility and performance, and the profiler traces from the previous section confirmed that the observed scaling behavior was dominated by GPU kernel execution rather than by data loading bottlenecks.

4.4 Gradient Accumulation

Following the batch sizing experiment, we turned to gradient accumulation and its impacts on throughput and resource utilization. For this test, the per-device batch size was fixed to 2 and the sequence length fixed to 2048 because it appeared to provide the best balance between memory footprint and performance. Gradient accumulation steps were varied across $\{1, 2, 4, 8\}$ and all other training parameters were kept constant.

Across accumulation sizes 1-4, GPU utilization was consistently high at around 97% and for all configurations the memory usage remained stable at 22.4 GB. Runtime decreased slightly as accumulation

Gradient Accumulation Steps	GPU Util (%, Mem)	Runtime	Samples/s
1	96.0%, 22.4 GB	623.35	1.604
2	97.0%, 22.4 GB	619.08	1.615
4	97.4%, 22.4 GB	614.63	1.627
8	88.8%, 22.4 GB	628.86	1.590

Table 4.5: Effect of gradient accumulation steps on GPU utilization, runtime, and throughput.

increased from 1 to 4, resulting in a modest throughput improvement as well. This suggests that moderate gradient accumulation tends to write off the initial cost of optimizer overhead and improves computational efficiency without significantly altering GPU execution behavior.

However, increasing the accumulation size to 8 led to a drop in average GPU utilization (89%) and a decrease in throughput. Runtime also increased relative to smaller accumulation sizes. This behavior indicated that an excessively large number of accumulation steps reduced the frequency of optimizer updates, leading to the introduction of idle periods between backward passes, in turn limiting GPU resource use.

4.5 Activation Checkpointing

Due to out-of-memory errors occurring when activation checkpointing was switched off, the sequence length had to be reduced to 1024 tokens and the per-device batch size fixed to 1 for this experiment. For clarity of trace interpretation, gradient accumulation was also set to 1. Both configurations converged to the same training loss (1.134) after one epoch, which allowed for performance comparisons without needing to worry about optimization effects on training quality.

Activation Checkpointing	GPU Util (%, Mem)	Runtime	Samples/s
on	76.6%, 18.4 GB	554.84	1.802
off	73.0%, 37.2 GB	412.12	2.426

Table 4.6: Effect of activation checkpointing on GPU utilization, runtime, and throughput.

Straying from the Apertus-provided default settings by disabling activation checkpointing immediately increased memory usage by over 100%. This confirmed that when checkpointing was enabled, intermediary activations were no longer being stored during the forward pass but rather were being recomputed during the backward pass. This technique enables training under tighter memory constraints but also comes with a clear cost. When gradient checkpointing was enabled, the runtime decreased by 35% and the throughput by 26%. The GPU utilization remained similar, however, pointing to the slowdown not being caused by stall in GPU work, but by additional time being added in the backward pass to recompute the previously discarded activation function outputs.

This can be seen in profiling traces; when checkpointing was enabled, a larger proportion of each training step was spent inside the backward pass, particularly within `torch/autograd/graph.py`’s `engine_run_backward` function (Figures 4.16 and 4.17). Compared to the non-checkpointed run, the backward pass spans a longer region of the timeline. This explains why loss computation occupies a smaller portion of the step time with checkpointing enabled: the backward pass increases, increasing training step time, rather than the loss computation itself becoming faster.

Importantly, no additional synchronization gaps or CPU-side bottlenecks were observed in either trace. Kernel execution remained dense, showing that checkpointing introduced extra compute work, not inefficiency.

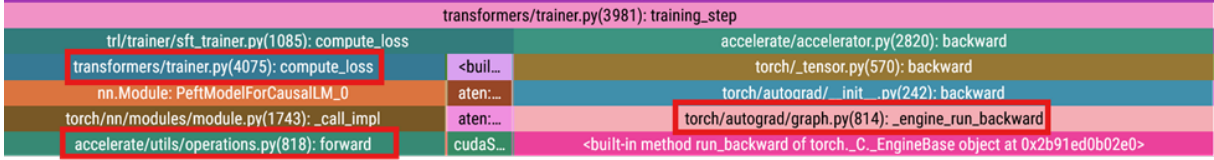


Figure 4.16: A high-level view of a single training step in the case where gradient checkpointing is not being used. The blocks represent Python functions, with blocks above serving as the calling entry point to blocks directly below them. Time is represented along the horizontal axis, with block widths representing execution time. Note the proportionally wider backward pass in Figure 4.17 compared to its forward pass. This is due to the additional gradient recomputation being performed at that stage.

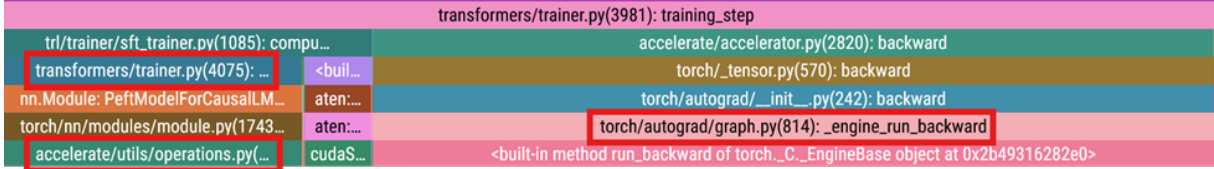


Figure 4.17: The same training step in the case when gradient checkpointing is being used. More time is proportionally spent in the backward pass here compared to Figure 4.16 due to extra computation being performed. End-to-end training step time is longer when gradient checkpointing is turned on.

4.6 Two-GPU Data Parallel Fine-Tuning with LoRA

Based on earlier experiments, the sequence length, batch size, and gradient accumulation were fixed to values that provided a good balance between throughput and memory usage in the single-GPU setting. The focus of this experiment was therefore on the scalability and overheads introduced by different distributed training strategies.

Two sets of experiments were conducted. First, DistributedDataParallel (DDP), ZeRO-2, and ZeRO-3 were compared without gradient checkpointing, as DDP kept throwing errors when enabled in this setup. All configurations used a sequence length of 1024, per-device batch size of 1, and no gradient accumulation, as that is what was able to fit within the memory constraints posed by having gradient checkpointing disabled. In the second experiment, gradient checkpointing was reenabled and ZeRO-2 and ZeRO-3 were reevaluated using a larger effective batch size (sequence length 2048, per-device batch size 2, gradient accumulation 4); these were the best-performing parameters identified in prior experiments.

4.6.1 First Experiment

Data-Parallel Strategy	GPU 0 Util (% , Mem)	GPU 1 Util (% , Mem)	Runtime	Samples/s
DDP	75.8%, 37.5 GB	84.5%, 37.9 GB	244.47	4.090
ZeRO-2	73.8%, 33.9 GB	77.7%, 34.4 GB	227.28	4.400
ZeRO-3	94.8%, 33.0 GB	96.1%, 33.0 GB	1421.28	0.704

Table 4.7: Effect of data-parallel training strategies on GPU utilization, runtime, and throughput in a two-GPU, single-node environment.

In the no-checkpointing configuration, both DDP and ZeRO-2 achieved similar runtime and throughput. The configurations exhibited a fairly balanced level of utilization between both GPUs, with ZeRO-2 reducing per-GPU memory footprint by around 3-4 GB. These results show that, for LoRA fine-tuning at this scale of device distribution, ZeRO-2 can provide effective memory savings over standard data parallelism.

In contrast, ZeRO-3 landed in with a dramatically different performance profile. Although the GPU utilization was above 90% on both devices, the runtime increased sixfold, resulting in a sub-sample per second throughput. This suggests the GPU utilization was not directly translating to effective training progress here. Torch profiler traces (Figures 4.18-4.20) revealed increased time spent in communication and parameter partitioning, which is typical of the sharding behavior of ZeRO-3. For a LoRA-based

workload, where the number of trainable parameters is limited to reduce memory pressure, these overheads completely dominated and outweighed the benefits of full parameter, gradient, and optimizer sharding that is on offer with ZeRO-3.



Figure 4.18: The CUDA stream for a single step of Apertus fine-tuning using DDP. The thin, multicolored bands at the top represent CUDA kernels performing actual training work. The bands circled in red are all_reduce communication calls, and the thin magenta band to the left is an all_gather communication call. These communication calls are limited, and the GPU is effectively saturated.

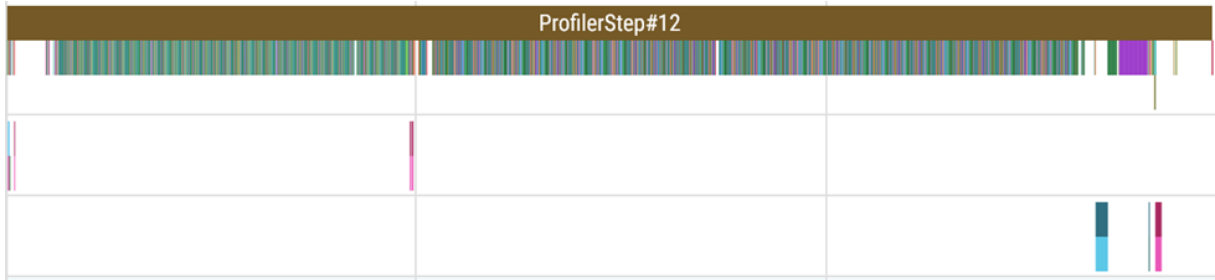


Figure 4.19: The CUDA stream for a single step of training using ZeRO-2. Like with DDP, the CUDA stream is healthily saturated with training kernels, and the training step is clearly GPU compute-bound.

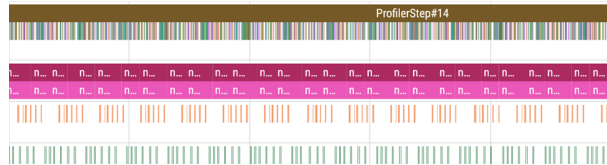


Figure 4.20: A snapshot of the CUDA stream for ZeRO-3 training taken over 500 milliseconds. The wide, continuous magenta and pink bands represent all_gather communication calls. The thin orange bands below them represent CatArrayBatchedCopy calls, which are used to copy tensor data between the CPU and GPU. The thin green bands at the bottom represent MemCpy calls from GPU to GPU, reflecting the sharding behavior of ZeRO-3.

4.6.2 Second Experiment

When gradient checkpointing was enabled, overall runtime increased for both ZeRO-2, marking the signature recomputation that gets introduced during the backward pass (Table 4.8). However, checkpointing significantly reduced memory usage, allowing the use of larger sequences and effective batch sizes. Under these conditions, ZeRO-2 again demonstrated superior performance, though the relative performance gap narrowed compared to the non-checkpointed case.

Profiler traces for the checkpointed runs showed largely the same CUDA stream characteristics as the prior experiment. While checkpointing reduced memory pressure, it did not mitigate the fundamental communication overhead introduced by full parameter sharding. As a result, ZeRO-3 remained inefficient for this workload.

Overall, these results indicate that for multi-GPU LoRA fine-tuning on a single node, ZeRO-2 provides the best trade-off between memory efficiency and training throughput. DDP performs competitively but lacks the flexibility of ZeRO-2 for using gradient checkpointing, and ZeRO-3 incurs overheads that make it less useful when training with LoRA on a single node.

Data-Parallel Strategy	GPU 0 Util (% , Mem)	GPU 1 Util (% , Mem)	Runtime	Samples/s
ZeRO-2	68.1%, 23.2 GB	96.6%, 23.7 GB	426.72	2.343
ZeRO-3	84.1%, 25.5 GB	98.8%, 25.3 GB	793.73	1.260

Table 4.8: Effect of data-parallel training strategies on GPU utilization, runtime, and throughput in a two-GPU, single-node environment.

4.7 Full-Parameter Fine-Tuning with Optimizer Offloading

As a final experiment, full-parameter fine-tuning was done to assess whether it could be made feasible on a single node with two 40 GB GPUs. Unlike the LoRA experiments, full fine-tuning exceeded GPU memory limits under all feasible configurations, requiring optimizer state offloading to CPU memory. Experiments were therefore conducted using DeepSpeed ZeRO-3 with optimizer offloading, and a comparative ZeRO-2 configuration under the same parameters.

Two configurations were evaluated, one for each the ZeRO-2 and ZeRO-3 optimizer strategies. Both used a sequence length of 2048 with batch size 2 and gradient accumulation 4. These configurations were selected to balance memory constraints against effective batch size, based on prior findings.

Data-Parallel Strategy	GPU 0 Util (% , Mem)	GPU 1 Util (% , Mem)	Runtime	Samples/s
ZeRO-2	62.2%, 26.0 GB	77.2%, 26.5 GB	3915.61	0.255
ZeRO-3	72.1%, 28.3 GB	81.5%, 27.5 GB	3606.12	0.277

Table 4.9: Effect of data-parallel full-parameter training strategies on GPU utilization, runtime, and throughput in a two-GPU, single-node environment.

With both strategies and optimizer offloading enabled, full fine-tuning was successfully executed in all cases, which confirmed the feasibility of this approach on the available hardware.

For both strategies, full fine-tuning proved workable when optimizer states were offloaded to the CPU. Unlike the LoRA experiments, ZeRO-3 proved itself to be the better performing option. ZeRO-3 produced the higher throughput in this experiment while maintaining a higher GPU utilization, suggesting the increased communication and synchronization overhead as seen in the profiling traces from the previous experiment became relevant performance indicators for full parameter training—as the number of trainable model parameters increased compared to LoRA training, ZeRO-3’s ability to shard said parameters became a performance benefit, rather than bottleneck.

Overall, these results demonstrated that full-parameter fine-tuning is technically achievable on limited hardware using optimizer offloading, but at a substantial performance cost. Compared to LoRA-based fine-tuning, throughput is reduced by more than an order of magnitude, and scaling efficiency is limited by the overhead incurred from communication and offloading rather than the raw compute capacity.

Chapter 5

Conclusion

This project was an investigation into how PyTorch employs hardware parallelism and how it could be seen in its execution of a deep learning workload, namely for Apertus fine-tuning. This investigation was originally motivated by how abstracted away the low-level execution details are to PyTorch users, and through systematic experimentation via detailed profiling, that abstraction has been reduced.

These experiments investigated multiple forms of parallelism, including asynchronous GPU execution, CPU-side multiprocessing, memory-compute tradeoffs, and multi-GPU data parallelism. For the Apertus fine-tuning workload, profiling traces showed that performance is governed by GPU kernel execution rather than by CPU-side data loading. In the data pipeline sensitivity experiment, it was found that while increasing DataLoader parallelism changed CPU utilization and execution structure, it did not appreciably improve training throughput.

Meanwhile, experiments involving batch size, sequence length, and gradient accumulation showed that higher GPU utilization does not necessarily improve training throughput either. Moderately increasing batch sizes and sequence lengths introduced more memory pressure on the GPU while keeping training time relatively stable and increasing these values too far resulted in extended kernel runtimes that ultimately reduced performance. Activation checkpointing was shown to reduce memory usage very effectively, but at an added cost to training time due to extra computation being introduced in the backward pass.

In the multi-GPU scenario, the main highlight came with making sure to match distributed training strategies (DDP, ZeRO-2, ZeRO-3) to the structure of the workload. For LoRA-based fine-tuning on a single, two-GPU node, ZeRO-2 proved to be the best-performing option. ZeRO-3, on the other hand, introduced an excessive amount of communication and synchronization calls that both increased GPU utilization and decreased training time.

Overall, the key takeaway of this work should be that the understanding of PyTorch performance requires analysis from more than just runtime utilization metrics. Profiling traces helped immensely with distinguishing what was productive computation and what was not, considering ZeRO-3's slower runtime and higher average utilization costs. In addition, by tying the observations made in these traces back to known parallelism mechanisms, this project demonstrated how profiling can be an effective explanatory tool for frameworks as abstracted as PyTorch. The insights gained here can be applicable to wide range of other PyTorch-based workloads.

Future Work

Although an additional reinforcement learning-based application was originally planned as part of this project, it could not be completed due to time and hardware constraints. However, implementing and profiling such a workload in future work would be a good to get an understanding of the PyTorch execution model from a complementary perspective; reinforcement learning differs from large-scale fine-tuning in its high-level execution model, where one the former prioritizes rewards for good answers and the latter labelled examples for specific tasks. Future work from this perspective could show variations in low-level execution behavior that would further support the findings in this work.

Bibliography

- [1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, and B. Steiner, “Pytorch: An imperative style, high-performance deep learning library,” *arXiv preprint*, vol. 1912.01703, 2019. arXiv:1912.01703.
- [2] PyTorch Documentation, “torch.utils.data.dataloader — pytorch documentation.” <https://docs.pytorch.org/docs/stable/data.html>, 2025. Accessed 2025.
- [3] PyTorch Tutorial Authors, “Getting started with distributed data parallel — pytorch tutorials.” https://docs.pytorch.org/tutorials/beginner/ddp_series_theory.html, 2019. Accessed 2025.
- [4] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimizations toward training trillion parameter models,” *arXiv preprint*, vol. 1910.02054, 2019. arXiv:1910.02054.
- [5] I. Svogor, C. Eichenberger, M. Spanring, M. Neun, and M. Kopp, “Profiling and improving the pytorch dataloader for high-latency storage: A technical report,” *arXiv preprint*, vol. 2211.04908, 2022. arXiv:2211.04908.
- [6] Y. Hao, X. Zhao, B. Bao, D. Berard, W. Constable, A. Aziz, and X. Liu, “Torchbench: Benchmarking pytorch with high api surface coverage,” *arXiv preprint*, vol. 2304.14226, 2023. arXiv:2304.14226.

Use of AI tools

AI-Based tool	Type of use	Affected parts of the work	Remarks
ChatGPT (OpenAI)	LaTeX syntax generation for tables	Ch. 3 and Ch. 4	



University
of Basel

Faculty of Science



Declaration on Scientific Integrity

(including a Declaration on Plagiarism and Fraud)

Translation from German original

Title of Thesis: Analyzing Parallelism and Performance in PyTorch

Name Assessor: Prof. Dr. Florina M. Ciorba

Name Student: Scott Barcomb

Matriculation No.: 24-061-640

I attest with my signature that I have written this work independently and without outside help. I also attest that the information concerning the sources used in this work is true and complete in every respect. All sources that have been quoted or paraphrased have been marked accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using the appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or expulsion from the study program.

Place, Date: Basel, 19.01.26 Student: 

Will this work, or parts of it, be published?

☒ No

☐ Yes. With my signature I confirm that I agree to a publication of the work (print/digital) in the library, on the research database of the University of Basel and/or on the document server of the department. Likewise, I agree to the bibliographic reference in the catalog SLSP (Swiss Library Service Platform). (cross out as applicable)

Publication as of: _____

Place, Date: _____ Student: _____

Place, Date: _____ Assessor: _____

Please enclose a completed and signed copy of this declaration in your Bachelor's or Master's thesis.